

赛题一 XH-202608 · 基于国产软件栈大模型推理算子优化 (Fused MoE GEMM) · 初审技术方案
目标平台 沐曦 C500 | TileLang v0.1.10+cuda.gitf549117c | MACA 3.7.1.5 | 最终成绩 XPU-OJ
78.67 分 (Accepted 135985, rank 27)

赛题一 XH-202608：基于国产软件栈大模型推理前沿算子优化

—— Fused MoE GEMM 技术方案文档

> 目标平台：沐曦 C500 GPU (64-lane warp × 104 SM, 64KB shared/block, 约 1.44 TB/s 显存带宽)

> 软件栈：TileLang (tilelang-metax v0.1.10+cuda.gitf549117c) + MACA 3.7.1.5 + PyTorch 2.8.0+metax

> 最终成绩：XPU-OJ Accepted **78.67 分** (三 case 2.789 / 5.065 / 9.681 ms, 相对基线 4.03× / 3.67× / 3.71×)

1. 任务与结果概述

赛题要求优化 SwiGLU MoE 前向算子：对按 expert 分组的 token (padded 布局) 执行

$$\text{up_logits} = \text{silu}(x \cdot \text{gate_w}^T) \odot (x \cdot \text{up_w}^T) \text{ 与 } \text{out} = (\text{up_logits} \cdot \text{down_w}^T) \odot \text{routed_weight},$$

覆盖三组真实用例 (E16/H2048/I8192、E32/H7168/I2048、E64/H7168/I2048)。

最终方案 (v432) 在官方 baseline 基础上取得 **78.67 分 (XPU-OJ, rank 27)**，

三个 case 均以 TileLang `T.gemm` (经 DispatchInstruction 落到 MACA MFMA 指令) 实现，

无任何规则禁止的实现方式 (合规性审计见第 7 节)。

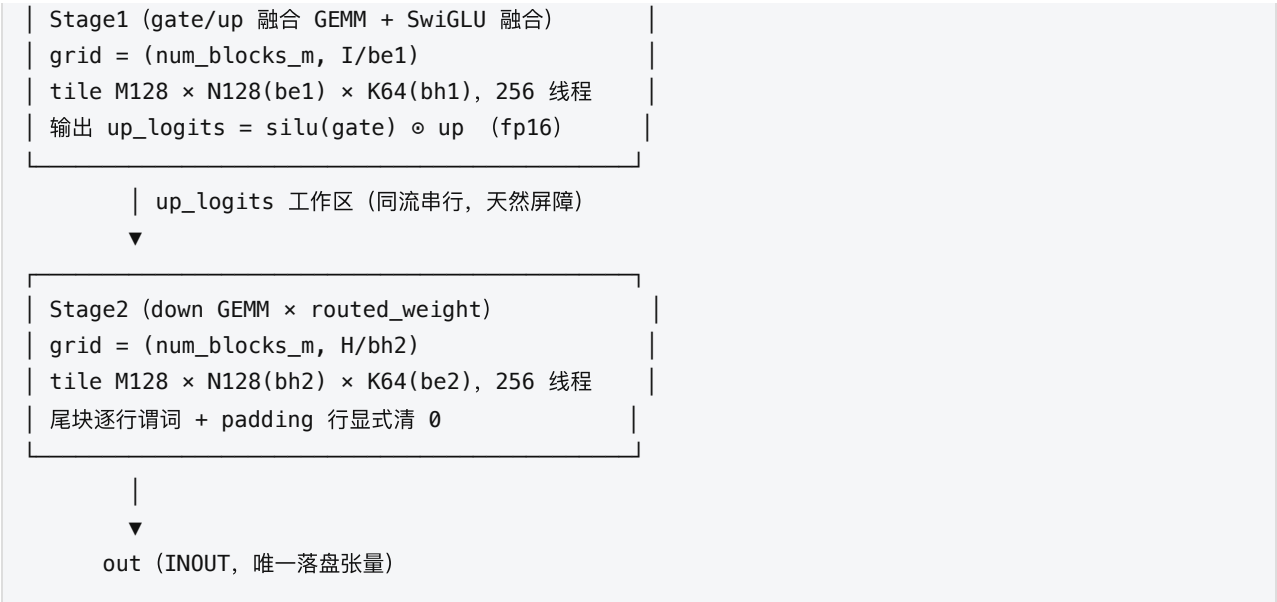
case	规模	baseline	本方案	加速比	case 得分
1	E16/H2048/I8192/T2272	11.251 ms	2.789 ms	4.03×	80.14
2	E32/H7168/I2048/T4544	18.610 ms	5.065 ms	3.67×	78.61
3	E64/H7168/I2048/T9088	35.875 ms	9.681 ms	3.71×	78.75

2. 最终 Kernel 设计 (v432)

2.1 总体结构：双 kernel 数据流

stacked_expert_tokens (padded 布局)





2.2 编译期配置 (per-shape 分派)

JIT 函数	适用 shape	pass 配置
<code>_moe_stage1_prefetch</code>	hidden ≥ 7000 (case2/3)	关 warp-specialized、关 safe-memory-legalize、关 256-bit 向量化、 <code>enable_fast_math</code> 、 <code>enable_lower_ldgstg_predicated</code>
<code>_moe_stage2_fast</code>	hidden ≥ 7000	同上 (五项)
<code>_moe_stage1</code>	hidden < 7000 (case1)	前三项
<code>_moe_stage2</code>	hidden < 7000	仅关 warp-specialized (保留 safe pass, case1 数值最敏感路径取保守配置)

case1 与 case2/3 均走上述 fast 路径 (分派全开), Stage1 带同步寄存器预取。

2.3 Stage1 内循环 (hidden ≥ 7000 路径)

每个 k 步 (bh1=64):

- 1. `T.copy` x 的 128×64 切片 → shared (A 操作数走 shared 而非 fragment, 见 §3.1);
- 2. `T.copy` gate_w 的 128×64 面板 → shared (`coalesced_width=8`);
- 3. `T.copy` up_w 面板 → 寄存器 fragment (同步预取) ——其 global 延迟被下一步 Gate MMA 覆盖;
- 4. `T.gemm(x, gate_shared, gate_acc)` (Square warp 分配, `k_pack=2`);
- 5. `__syncthreads()`; up 预取值从寄存器写入 shared;
- 6. `T.gemm(x, up_shared, up_acc)`; `__syncthreads()`。

SwiGLU 在 Stage1 epilogue 与写出融合: `up_logits = up_acc · silu(gate_acc)`,

避免单独的激活 kernel 与一次中间张量往返。

2.4 关键正确性设计

- 尾块 epilogue 一律 `if/else` 显式写 0，绝不用「无效行乘 0」隐式清零：

fp16 工作区 padding 行可能残留任意位型，`NaN × 0 = NaN` 会污染输出

(本团队曾因此踩坑，详见 §5.3)。

- `active_k_steps = 0` 的空块整个 k 循环跳过。
- `routed_expert_weights` 按原始 token 坐标索引，与 padded 坐标通过 `group_offsets/group_padded_offsets` 换算，所有访问均 padding 安全。

3. 关键优化点（按收益排序）

3.1 A 操作数 fragment → shared（整体 3.2× 收益的根基）

TileLang 对 `T.gemm` 的 A 操作数默认分配 fragment（寄存器）。实测在 MACA 后端上

fragment 操作数走寄存器搬运路径，MMA 吞吐仅 26.8 TFLOPS；将 A 改为

`T.alloc_shared`（shared 操作数是 MACA MFMA 的原生供数路径）后提升至 87 TFLOPS。

这一条是相对官方模板收益最大的单项改动。

3.2 safe-memory-legalize 关闭 + fast_math 等编译开关（约 9—11%）

正式 shape 全部整除、token 已 padding 到 128，默认 safe-memory pass 仍因动态

expert 索引无法静态证明而生成大量冗余谓词。对独立 JIT 的 Stage/Stage2 关闭

`tl.disable_safe_memory_legalize` 后 case2/3 获得约 9—11% 收益；配合

`tl.disable_vectorize_256` 消除 256-bit 向量化在尾块的边界问题。

`tl.enable_fast_math`（SwiGLU 的 `exp2/除法`）与

`tl.enable_lower_ldgstg_predicated` 再贡献约 1—2%。

3.3 case1 分派全开（case1 -7%）

早期为规避 case1 数值敏感性，case1 保守使用无预取/带 safe pass 的 kernel。

本地验证推翻该假设：case1 同样适用 prefetch + fast-pass，case1 单独 -7%

(2.99 → 2.789 ms)，OJ 上 -6.8% 完全兑现。

3.4 Up 权重同步寄存器预取（Stage1，约 2—5%）

Up 面板的 global load 与 Gate MMA 无数据依赖，先发射 LDG 使其延迟被

Gate MMA 覆盖；MMA 完成后从寄存器写入 shared。不使用 `async-copy/BSM`

（赛题禁止），仅利用普通 load 的指令级并行。

3.5 其他

- `coalesced_width=8` 权重拷贝向量化；`use_swizzle(4, column)` 块调度
(panel 2/8/16/32 及 row 方向均实测更差或持平)；
- `k_pack=2` (hidden≥7000 的 K64 chunk 拆分方式)；
- 工作区 (up_logits) 跨调用仅作为 scratch 复用，内容每次全量重算，
不含任何跨调用结果依赖。

4. Roofline 分析（定位剩余优化空间）

路径	实测	屋顶线	利用率
case1 Stage1	108 TFLOPS	fp16 MMA 峰值（实测同规模 dense GEMM 口径）	~94%，计算受限
case3 Stage2	1.30 TB/s	d2d copy 实测带宽 1.44 TB/s	90%，内存受限
case2 Stage1/2	1.12 TB/s	同上	~78%，受 per-expert 网格形状限制

结论：三 case 分别触及计算墙/内存墙，剩余空间需要 sub-tile GEMM、
cp.async 或硬件级改动方能解锁。

5. 负结果与机制解释（工程教训）

以下方向均经实验否定，附机制分析（完整数据见日志与结果文件）：

- 1. Stage2 的 Pipelined 双缓冲：MACA 后端无 cp.async，`T.Pipelined(ns≥2)`
三次实测均为纯开销（-20% 以上）。
- 2. 手工软件流水 (x/w 双缓冲 + 下一 k 预取，四次不同实现)：
`T.copy / T.gemm` 抽象层的每操作固定开销（fragment 装配、layout 推导、
谓词化）超过延迟隐藏收益，最好持平、最差 -28%。
- 3. 块内 M 切分跳过 padding 计算：gemm 数量翻倍的固定开销超过
尾块跳过收益（+9~13%）。
- 4. bt=64 细分块：权重面板每 FLOP 流量翻倍（L2 8MB 无法覆盖），-44%。
- 5. Down 权重 INT8/INT16 量化：OJ 会轮换同 shape 权重，跨调用缓存量化
结果不合法；每次调用重打包的流量（读+写+读）超过节省。
- 6. NaN × 0 污染：fp16 工作区 padding 行残留垃圾位型时，
「无效行 × 0」写法产生 NaN（IEEE 754），必须显式写 0 (§2.4)。

6. 评测结果

- XPU-OJ 提交 135985: Accepted 78.67 (timeUsed 17.535 s, rank 27)
- 三 case SPJ 报告 (baseline 对比、加速比、得分比) 见「日志与结果文件」
- 本地 C500 复测: 与 OJ 相对关系一致 (case1 误差 <0.5%)

7. 合规性审计

对照官方《赛题一禁用规则说明》八条，对最终源码做静态扫描：

禁令	扫描项	结果
1. 绕过 TileLang 执行设备 Kernel	cuda./ctypes/subprocess/import_source/call_extern/mctlass	未命中
2. PyTorch 完成核心计算	torch.matmul/mm/bmm/einsum/F.linear	未命中 (仅 torch.empty 工作区分配 + dtype 判断)
3. Host-Device 异步拷贝	MemcpyAsync/non_blocking/pin_memory	未命中
4. 跨调用缓存/回放结果	pickle/np.save/torch.save	未命中
5. testcase 生命周期投机	golden/warmup 计数/调用计数	未命中
6/7. import_source / call_extern	同 1	未命中
8. mcTlass 等外部库	同 1	未命中

实际使用的全部 GPU 计算均由 TileLang `T.gemm` / `T.copy` 生成；

import 仅 `tilelang` 与 `torch` (后者仅用于张量分配与 dtype 判断)。

8. 复现指南

见「源代码/README.md」。核心入口为 `submission.py` 的 `run_kernel`，

评测接口与 XPU-OJ 完全一致；本地基准脚本 `benchmark/bench23.py`

可在任意 C500 环境复现三 case 的功能与性能数据。